Kani in the LLM Stack

Where does it fit in the ecosystem?

- Four main layers of LM software

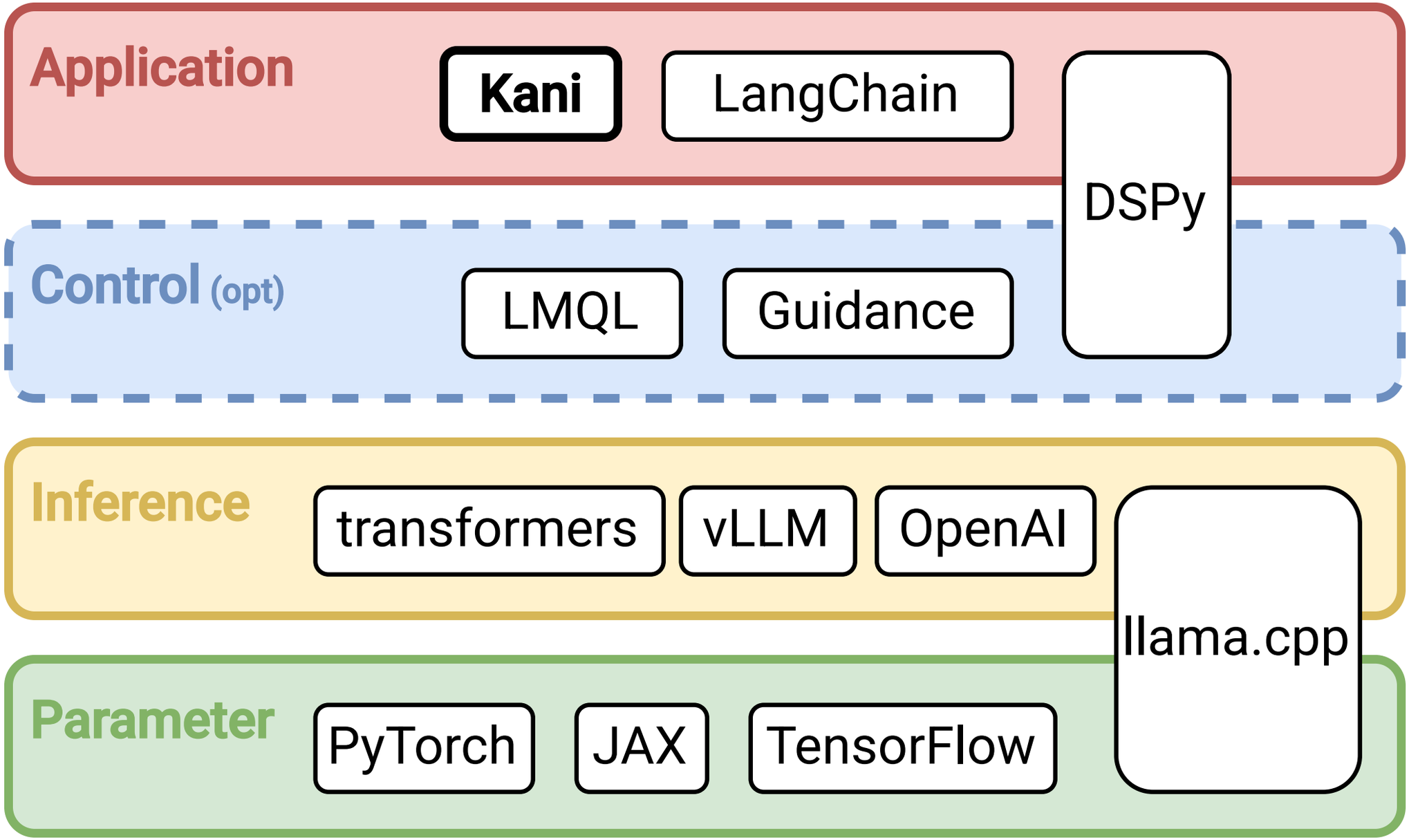
- **Parameter**: low-level weights, physical devices

- **Inference**: sampling strategies, tokenization

- **Control** (optional): guided decoding, dynamic prompt branching, structured output enforcement

- **Application**: chat history management, tool usage

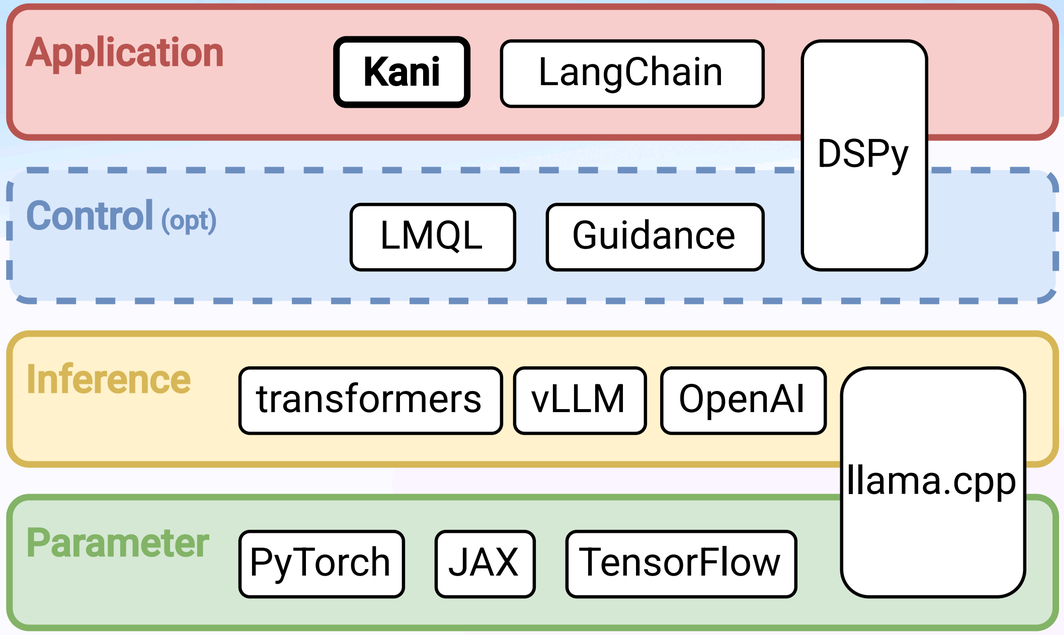
22


Application
Kani
LangChain
DSPy
Control (opt)
LMQL
Guidance
Inference
transformers
vLLM
OpenAI
llama.cpp
Parameter
PyTorch
JAX
TensorFlow

Kani in the LLM Stack

Where does it fit in the ecosystem?

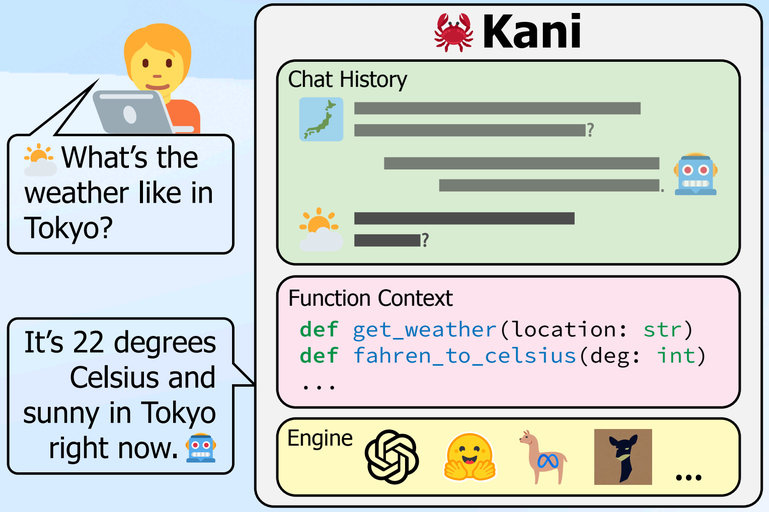
- Four main layers of LM software
- **Parameter**: low-level weights, physical devices
- **Inference**: sampling strategies, tokenization
- **Control** (optional): guided decoding, dynamic prompt branching, structured output enforcement
- **Application**: chat history management, tool usage



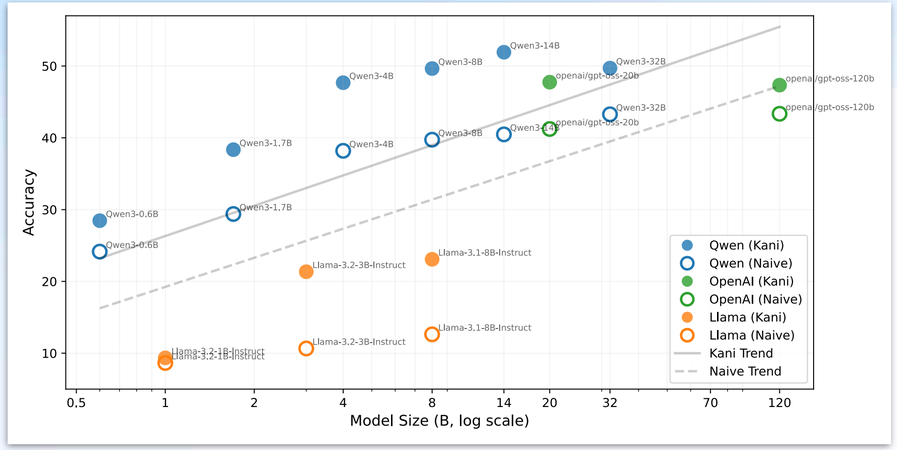
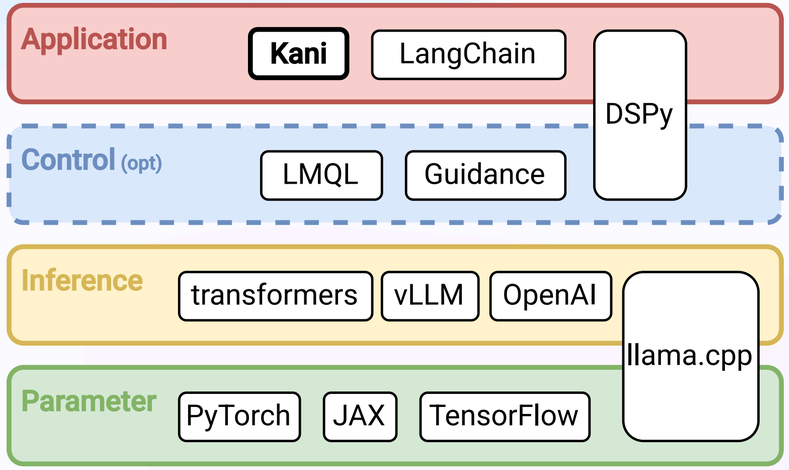
Kani Conclusions

A strong foundation for building LLM agents

Robust and Reproducible
Agent Primitives



Stability and Correctness
Tool Calling



Libraries and Standards
Compatibility